



Solving Batched Initial Value Problems on GPUs

Ben Longaker¹

April 30, 2026

¹The University of Texas at Austin, Oden Institute for Computational Engineering and Sciences

Initial Value Problems (IVPs)

$$\begin{cases} \dot{\mathbf{y}} = f(t, \mathbf{y}; \mathbf{p}), \\ \mathbf{y}(t_0) = \mathbf{y}_0, \end{cases}$$

$\mathbf{y}(t)$: state vector, $\mathbf{p}$: vector of fixed parameters. Goal: given many different $\mathbf{y}_0, \mathbf{p}$, compute $\mathbf{y}(t_f)$ for each system.

- Important application: astrochemistry. We need to solve a large IVP for each spatial grid cell, for each (outer loop) time step. This is a major computational bottleneck in these simulations; it is necessary to have surrogate/fast batched solver.

The two most simple time integrators are **forward** and **backward Euler**:

$$\mathbf{y}_{k+1} = \mathbf{y}_k + \Delta t \cdot f(t_k, \mathbf{y}_k; \mathbf{p}) \quad (\text{FE})$$

$$\mathbf{y}_{k+1} = \mathbf{y}_k + \Delta t \cdot f(t_k, \mathbf{y}_{k+1}; \mathbf{p}) \quad (\text{BE})$$

- Both extremely simple to implement, assuming the time step is fixed. (BE) requires a nonlinear equation solver. To parallelize: solve one ODE per thread.
- As $\Delta t \rightarrow 0$, the numerical solution approaches the true solution.
- These are examples of **first-order methods**: the error of the solution decreases at a linear rate.

Adaptive Time-Stepping

In most applications, we would like to change the time step to match the dynamics. This is done by considering two time-steppers of different order and comparing the difference. For example: **RK23** (Bogacki, Shampine 1997):

$$k_1 = f(t_k, \mathbf{y}_k)$$

$$k_2 = f\left(t_k + \frac{\Delta t}{2}, \mathbf{y}_k + \frac{\Delta t}{2}k_1\right)$$

$$k_3 = f\left(t_k + \frac{3\Delta t}{4}, \mathbf{y}_k + \frac{3\Delta t}{4}k_2\right)$$

$$\mathbf{y}_{k+1}^{(3)} = \mathbf{y}_k + \frac{\Delta t}{9}(2k_1 + 3k_2 + 4k_3)$$

$$t_{k+1} = t_k + \Delta t$$

$$k_4 = f\left(t_{k+1}, \mathbf{y}_{k+1}^{(3)}\right)$$

$$e_{k+1} = \frac{\Delta t}{72}(-5k_1 + 6k_2 + 8k_3 - 9k_4).$$

- If $\|e_{k+1}\| > \epsilon_{\text{tol}}$, we shrink the time step and recompute. In either case we compute a new time step to use for the next:

$$\Delta t^{(k+1)} = \Delta t^{(k)} * \left(\frac{\epsilon_{\text{tol}}}{\|e_{k+1}\|} \right)^{0.2}.$$

Interesting GPU problems:

- If each ODE is solved on a single thread, some ODEs may reach t_f faster than others. Hence many threads will remain idle while the others finish.
- SIMT will break: we would like similar time steps to take place on the same block.
- Solution: cluster the initial conditions and parameters. Sort these before launching, so that each warp contains ODEs that are similar to each other. This maximizes synchronization inside thread blocks.

Step 1: Build a very simple batched
IVP solver

The simplest case

- Explicit time-stepping (forward Euler), constant time step.
- Use Python **CuPy** interface, with **Raw Kernels** for solvers.
- To include custom RHS, embed text directly into the CUDA kernel (written in C) before compiling.
 - Best of both worlds: **Python** for user interface, low-level **C** makes code very efficient.
- Compare two different kernels:
 - `forward_euler_kernel`: on one thread, takes a single time step. Measure time for many steps.
 - `forward_euler_kernel_multistep`: on one thread, runs a single IVP to completion. Measure time for a single launch

Single Time Step vs. All at Once

We can see that it is much faster to do time-stepping **inside the kernel**.

Comparison to Serial Algorithm

Use numpy to solve Lorenz system.

- Same thing as before: one ODE per thread.
- Introduce thread divergence when we decide to accept/reject time step.
Ignore this for now.
- Otherwise each thread does the exact same thing. Some threads may finish before others, but this is fine.

